DAA By B.A. Khivsara
Unit 2

Page No.
Date

Analysis of algorithms & complexity Theory

Analysis

Input Size-

- Input size is no of elements in the i/p.
- Running time depends on input size
- Measure time efficiency as function of i/p size.
- Diff^n i/p of size n may take a diff^n amount of time.

How do we determine the i/p size?

1. Natural parameter.
2. For sorting & other problem on array - i/p size depends on Array size
3. For combinational problems: No of objects
4. For Graphs: no of vertices & edges.

imp is class of problem.

Types of analysis

- The function f(n), gives the running time of an algorithm depends not only on the size 'n' of the i/p data but also on the particular data
- The complexity function f(n) for certain cases are
  1. Best case — The minimum possible value of f(n) is called the best case.
     Best case complexity of the algo is the minimum function defined by the ~~maximum~~ no of steps taken on any instance of size n.
     - It provides a lower bound on running time
     - I/p is the one for which the algorithm runs the fastest.

SPPU ENGINEERING UPDATES

Study material provided by : Vishwajeet Londhe

Join community by clicking below links

@SPPU_ENGINEERING_UPDATE

Insta Page

https://www.instagram.com/sppu_engineering_update

@SPPU_TE_BE_COMP

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

WhatsApp Channel

https://whatsapp.com/channel/0029VaAhRMdAzNbmPG0jyq2x

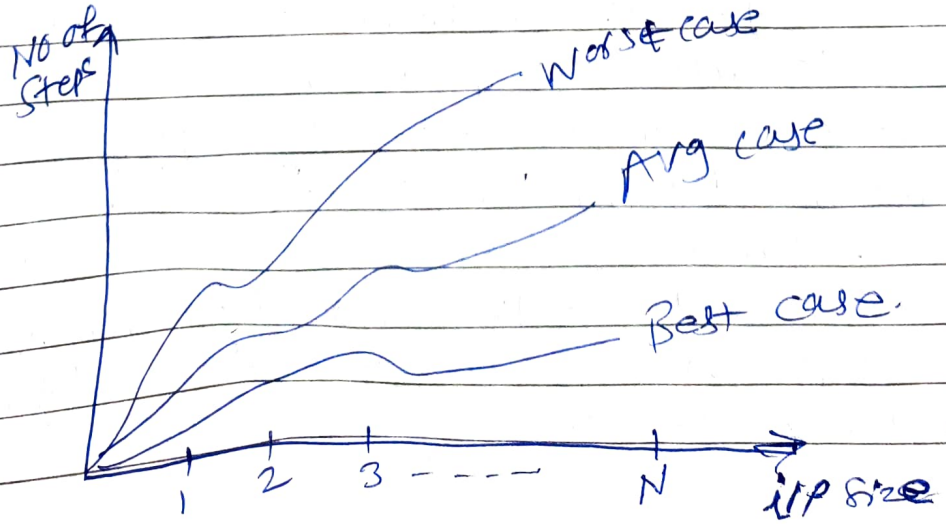
2. Worst case

- Worst case complexity of the algorithm is the function defined by the maximum no of steps taken on any instance of size n.
- Provides upper bound on running time.
- An absolute guarantee that the algorithm would not run longer, no matter what the inputs are.

3. Average case — The expected value of f(n).
- Average case complexity of the algorithm is the function defined by the average no of steps taken on any instance of size n.
- Provides a prediction about running time
- Assumes that the i/p is random.

Growth Rate:-

- It is defined as the rate at which running time increases as a function of i/p.
- The running time of an algorithm is the function of an i/p size given to an algorithm.



Unit II

Growth rate –
efficiency classes.

Class	order of Growth rate	Example
~~Constant~~ Constant	1	– Delete first node from link list – Add 2 nos
Logarithmic	log n	– Binary search
Linear	n	– Linear search – Find max/min no from array
n log n	n log n	Mergesort, Quick sort
Quadratic	n^2	Selection sort Bubble sort
Cubic	n^3	Matrix multiplication
Exponential	2^n	– Knapsack problem for optimal soln – TSP using dynamic Programming
Factorial	n!	– Generating permutation of given set. – TSP using brute force approach

Date

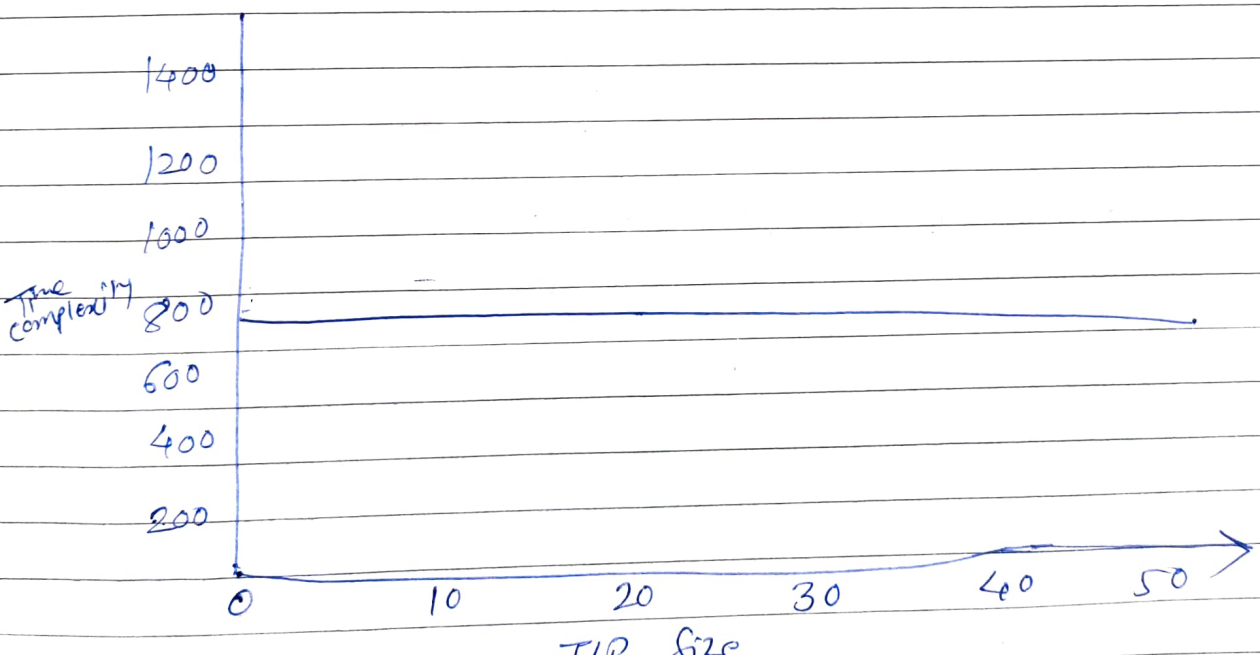
Growth Rate -

- It is defined as the rate at which running time increases as a function of i/p.

- The running time of an algorithm is the function of an i/p size given to an algorithm.

- eg.
  The cost of function
  $f = n^8, n^6, n^3$ - highest value is n^8.
  ie rate of growth.

- A variety of growth rates that are ~~represented~~ representative of typical algorithm are shown.

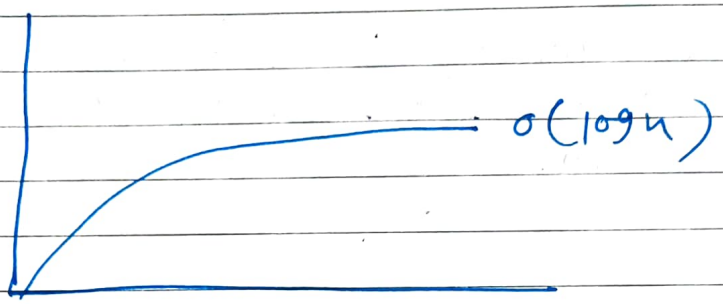


Constant Growth rate

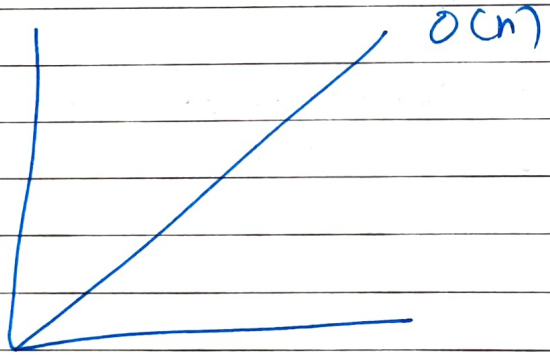
This means processing 1 piece of data takes the same amount of resource as processing 1 Million piece of data. Graph of such a growth rate looks like a horizontal line.

2) Logarithmic Growth Rate

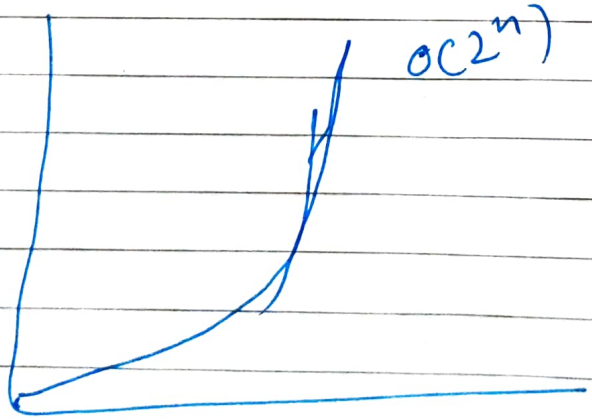
Logarithmic growth rate is a growth rate where the ~~real time need~~ required grows by one unit each time the data is doubled.



3) linear Growth rate



4) exponential Growth rate



Page No.
Date

Asymptotic Growth

- It is a mathematical way of representing the time complexity.
- To compare two algorithms with running times f(n) & g(n), we need a rough measure that characterizes how fast each function grows.
- Unless we evaluate the performance of an algorithm by taking a sufficiently large i/p size, we cannot justify its growth rate.
- Hence we study asymptotic growth of an algorithm where the performance of an algo is verified against the increasing i/p size boundlessly.

Types of Asymptotic Notations

1. Big "oh" (O)
2. Big Omega (Ω)
3. Big (Θ)
4. Little oh (o)
5. Little omega (ω)

O Notation - Asymptotic "less than"
$f(n) = O(g(n))$ implies $f(n)$ "$\leq$" $g(n)$

Ω Notation - Asymptotic "Greater than"
$f(n) = \Omega(g(n))$ implies $f(n)$ "$\geq$" $g(n)$

Θ Notation - Asymptotic "equality"
$f(n) = \Theta(g(n))$ implies $f(n)$ "$=$" $g(n)$

Page No.
Date

1) Big oh

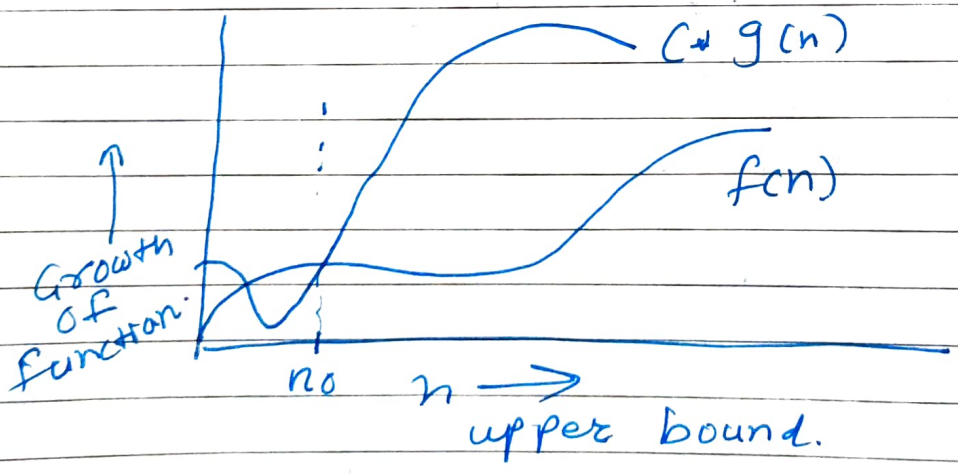
This notation is denoted by 'O' & it is pronounced as "Big oh". Big oh notation defines upper bound for the algorithm, it means the running time of algorithm cannot be more than its Asymptotic upper bound for any i/p data.

- Let $f(n)$ & $g(n)$ are two non-negative functions indicating running time of two algorithms.

- we say, $g(n)$ is upper bound of $f(n)$, if there exists some positive constants C and n_0 such that $0 \leq f(n) \leq C * g(n)$ for all $n \geq n_0$.

- It is denoted as

$$f(n) = O(g(n))$$



g(n) is an asymptotic upper bound for f(n).

upper bound.

eg.

1) $n^4 + 100n^2 + 10n + 50$
$= O(n^4)$

2) $n^3 - n^2$ is $O(n^3)$

eg.

Find upper bound of running time of linear function

$$f(n) = 6n + 3$$

Sol^{n} -

We have to find C and n_0 such that

$$0 \leq f(n) \leq C \cdot g(n) \quad \text{for all } n \geq n_0$$

$$0 \leq f(n) \leq C * g(n)$$

$$0 \leq 6n + 3 \leq C * g(n)$$

$0 \leq 6n + 3 \leq 6n + 3n$, for all $n \geq 1$

(there can be such infinite possibilities)

$0 \leq 6n + 3 \leq 9n$ for all $n \geq 1$

so $C = 9$ & $g(n) = n$, $n_0 = 1$

Tabular approach

$$0 \leq 6n + 3 \leq C * g(n)$$

$$0 \leq 6n + 3 \leq 7n$$

Now manually find out the proper n_0, such that

$$f(n) \leq C \cdot g(n)$$

n	f(n) = 6n+3	C·g(n) = 7n
1	9	7
2	15	14
3	21	21
4	27	28
5	33	35

from above table we can say

1) $f(n) = O(g(n)) = O(n)$
   for $C = 9$ $n_0 = 1$

2) $f(n) = O(g(n)) = O(n)$
   for $C = 7$, $n_0 = 3$.

& so on --- There can be multiple pair of (C, n_0) possible.

eg 2) Find upper bound of $f(n) = 3n^2 + 2n + 4$

(2) Big Omega

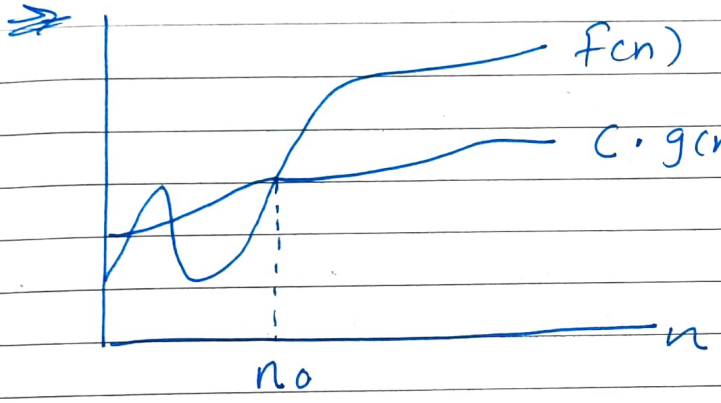
- This notation is denoted by 'Ω' & it is pronounced as "Big omega".
- Big Omega notation defines lower bound for the algorithm.
- It means the running time of algorithm cannot be less than its asymptotic lower bound for any random sequence of data.

We say,

→ $g(n)$ is lower bound of $f(n)$ if there exist some positive constants c and n_0, such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$, ~~such~~

→ It is denoted as

$$f(n) = \Omega\, g(n).$$



$g(n)$ is an asymptotic lower bound for $f(n)$.

eg. 1) Find lower bound of running time of linear function $f(n) = 6n + 3$

Soln We have to find c and n_0 such that

$$0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0$$

$$0 \leq c \cdot g(n) \leq f(n)$$

$$0 \leq c \cdot g(n) \leq 6n+3$$

$$0 \leq 6n \leq 6n+3 \quad -①$$

$$0 \leq 5n \leq 6n+3 \quad -②$$

Above both ~~equal~~ inequality are true & there exists such infinite ~~en~~ inequalities. So

1) $f(n) = \Omega(g(n)) = \Omega(n)$ for $c = 6 \& n_0 = 1$

2) $f(n) = \Omega(g(n)) = \Omega(n)$ for $c = 5$, $n_0 = 1$

Page No.
Date

eg 2) Find lower bound of running time of quadratic function

$$f(n) = 3n^2 + 2n + 4$$

Soln

We have to find C and n_0 such that $0 \leq C \cdot g(n) \leq f(n)$ for all $n \geq n_0$

$$0 \leq C \cdot g(n) \leq f(n)$$
$$0 \leq C \cdot g(n) \leq 3n^2 + 2n + 4$$
$$0 \leq 3n^2 \leq 3n^2 + 2n + 4, \text{ for all } n \geq 1$$
$$0 \leq n^2 \leq 3n^2 + 2n + 4, \text{ for all } n \geq 1$$

Above both inequalities are true and there exists such infinite inequalities. So,

$$f(n) = \Omega(g(n)) = \Omega(n^2)$$
for $C = 3$, $n_0 = 1$

$$f(n) = \Omega(g(n)) = \Omega(n^2)$$
for $C = 1$, $n_0 = 1$ & so on.

eg 3) find lower bound of running time of quadratic function.

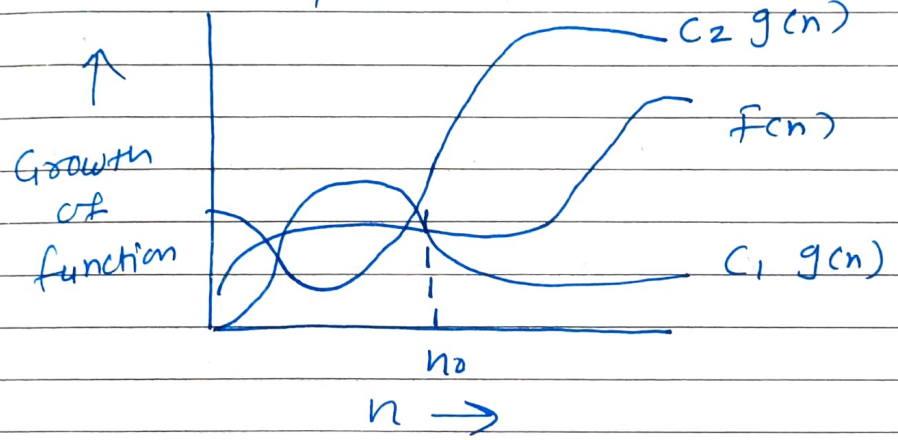
$$f(n) = 2n^3 + 4n + 5.$$

Page No.
Date

(3) Big Theta Notation

- This notation is denoted by 'Θ' and it is pronounced as "Big Theta". Big Theta notation defines tight Bound for the algorithm.

- It means the running time of algorithm cannot be less than or greater than it's asymptotic tight bound for any random sequence of data.



We say, $g(n)$ is tight bound of $f(n)$ if there exist some positive constants c_1, c_2 & n_0 such that

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

for all $n \geq n_0$.

It is denoted as

$$f(n) = \Theta(g(n))$$

eg - Find tight bound of running time of constant function $f(n) = 23$

Soln
- we have to find c_1, c_2 and n_0 such that

$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$

$0 \leq c_1 g(n) \leq 23 \leq c_2 g(n)$

$0 \leq 22 \cdot (1) \leq 23 \leq 24 \cdot (1)$ for all $n \geq 1$

So

$(c_1, c_2) = (22, 24)$

$g(n) = 1$

$n_0 = 1.$

$\therefore f(n) = \Theta(g(n)) = \Theta(1)$ for $c_1 = 22$, $c_2 = 24$ $n_0 = 1$

(2) Find tight bound of $f(n) = 6n + 3$

Soln - We have to find c_1, c_2, n_0 such that

$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$

$0 \leq c_1 g(n) \leq 6n + 3 \leq 6 c_2 g(n)$

$0 \leq 5n \leq 6n + 3 \leq 9n$ for all $n \geq 1$

$\therefore f(n) = \Theta(g(n)) = \Theta(n)$

for $c_1 = 5$ $c_2 = 9$ $n_0 = 1$

(3) Find tight bound for

$f(n) = 3n^2 + 2n + 4$

Page No.
Date

(4) Little oh (o)

→ Big-oh notation may or may not be tight.
→ It is also called loose upper bound.
→ Little oh is used to define bound which is asymptotically not tight.

eg.

→ Let f(n) & g(n) are the functions that map positive real numbers.

→ f(n) grows slower than g(n)

eg.

$$\text{if } \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

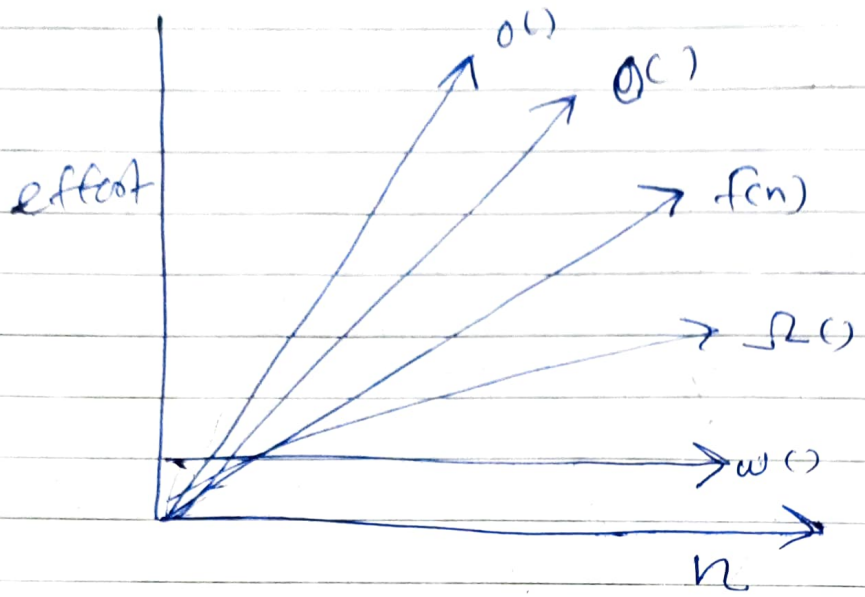
formally Stated as $f(n) = o(g(n))$

(5) Little Omega – (ω)

→ Big Omega notation may ~~not~~ or may not be tight.
- But Little omega is used to define bound which is asymptotically not tight.
- f(n) grows faster than g(n)

$$\text{if } \lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

formally Stated as
$f(n) = \omega(g(n))$


O()
Θ()
f(n)
Ω()
ω()
effect
n

Page No.
Date

* Polynomial and Non polynomial problems

There are two categories of ~~algorithm~~ problems.

1) Polynomial → Problems that can be solved by a polynomial time algorithm.

eg. Searching $O(\log n)$
Sorting $O(n \log n)$
~~Sort~~ String editing $O(mn)$

2) Non polynomial → problems for which no polynomial time algorithm is known

eg. Traveling Salesperson $O(n^2 2^n)$
Knapsack Problem $O(2^{n/2})$.

* Deterministic algorithm →

→ Result of every operation is uniquely defined.
→ For a particular i/p computer will give always same output.
→ Can solve problem in Polynomial time
→ Can determine the next step of execution

eg.

$5 + 3 = 8$ } Always produce the same o/p

* Non deterministic algorithm →

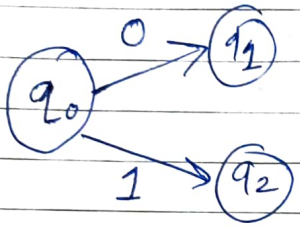
- For same i/p, compiler may produce diff^n o/p in diff^n runs.
- Cannot solve problem in Polynomial time
- Cannot determine the next step of execution due to more than one path the algo. can take

eg

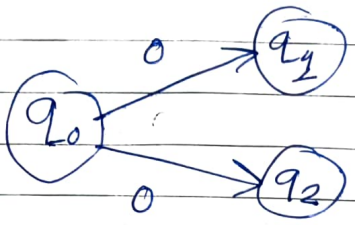
$a + b = 21.$

- Which 2 number sum is equal to 21.
- Many possibility;

1 + 20
2 + 19
3 + 18
⋮



Deterministic algo



Non deterministic algo

To specify non-deterministic algo, some terms are defined are as follows:

1) choice (x) — chooses any value randomly from set x.
2) Failure () - denotes the unsuccessful soln
3) Success() - denotes a successful soln.

Example -

Problem statement - Search an element x on A[1..n] where n≥1, on successful search return j, if A[j] equals to x, otherwise return 0.

Non deterministic algo for this problem =>

```
j = Choice (1,n);
if (A[j]==x) then
{ write (j);
  success();
}
write (0); failure();
```

Page No.
Date

* Decision Problem & optimization Problem

1) Decision problem →
- Any problem for which the answer is either zero or one (yes or no) is called a decision problem.
- An algorithm for a decision problem is termed a decision algorithm.

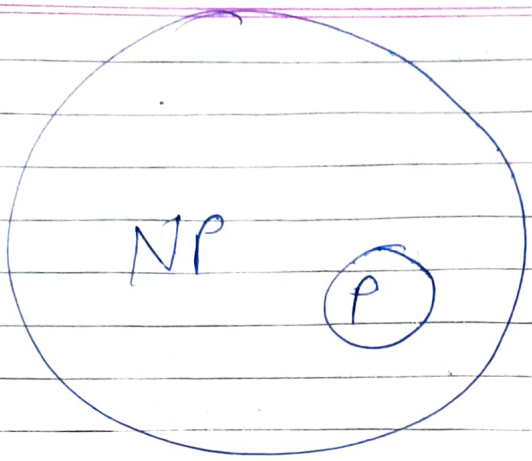
2) optimization problem →
- Any problem that involves the identification of an optimal (either maximum or minimum) value of a given function is known as an optimization problem.
- An optimization algorithm is used to solve an optimization problem.

Decision problem is assumed to be easier to solve compared to the optimization problem.

* P, NP, NP hard, NP complete :-

→ Defⁿ of P – Problems that can be solved in polynomial time. ('P stands for polynomial). using deterministic algo.
eg. Searching, sorting etc.

→ NP – It stands for non-deterministic polynomial time. It is a collection of decision problems that can be solved by a non-deterministic machine in polynomial time.



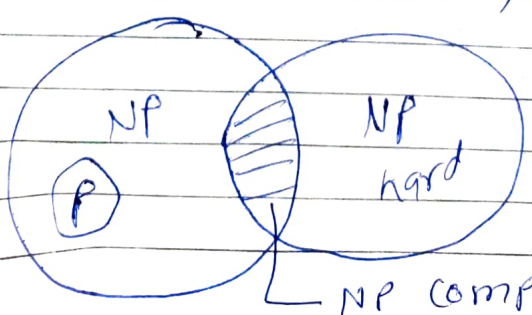
Since deterministic algo are just a special case of non-deterministic ones, we conclude that $P \subseteq NP$.

- The NP class problems can be further categorized into NP-complete & NP-hard problems.

- A problem A is called NP-complete if
  1. It belongs to class NP.
  2. Every other problem B in NP can be transformed to A in polynomial time.
     ie. For every $B \in NP$,
     $B \leq_p A$.

- These 2 facts prove that NP-complete problems are the harder problems in class NP. They are often refered to as NPC.

- However a solution of any NP-complete problem can be verified in polynomial time, but cannot be obtained in polynomial time



Page No.
Date

* P vs NP - differentiate betn P & NP problem/algo

P problem	NP problem
1. P problems are set of problems which can be solved in polynomial time by deterministic algo.	NP problems are the problems which can be solved in non-deterministic polynomial time.
2. P problems can be solved & verified in polynomial time.	Solution to NP problems cannot be solved/obtained in polynomial time, but if the soln is given, it can be verified in polynomial time.
3 P problems are subset of NP problems.	NP problems are Superset of P problems.
4. All P problems are deterministic in Nature.	All NP problems are non-deterministic in nature.
5 eg. Selection sort, Linear search	eg. TSP, Knapsack problem.

NP hard VS NP complete problems

NP hard	NP complete
1. NP hard problem (say A) can be solved if & only if there is NP complete problem (say B) can be reducible into A in polynomial time	NP complete problems can be solved by deterministic algorithm in polynomial time.
2 To solve NP hard problem, it must be in NP class	To solve NP complete problem, it must be in both NP & NP hard problems.
3. It is not a decision problem	It is a decision problems.
4. For eg. Halting problem, Hamiltonian cycle problem	eg. Determine whether a graph has a Hamiltonian Cycle.

* Polynomial Problem Reduction

To prove whether particular problem is NP complete or not we use polynomial time reducibility. That means if

$A \xrightarrow{poly} B$ and $B \xrightarrow{poly} C$

then $A \xrightarrow{poly} C$

→ Let L_1 and L_2 be two problems.

→ problem L_1 reduces to L_2 (also written as $L_1 \propto L_2$) if and only if there is a way to solve L_1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L_2 in polynomial time.

→ This defn implies that if we have a polynomial time algorithm for L_2, then we can solve L_1 in polynomial time.

→ Reduction in NP completeness takes one of ~~three forms~~ these forms -

~~1. Restriction~~

1. Local Replacement
2. Component Design

~~2.~~ Local replacement — In this reduction A ~~&~~ B by dividing i/p to A in the form of components and then these components can be converted to components of B.

Component Design — In this reduction A $\propto$ B by building special component for i/p B that enforce properties required by A.

→ Two problem P & Q are said polynomially equivalent if & only if P $\propto$ Q and Q $\propto$ P.

→ If problem P_2 is NP- Complete & there is polynomial time reduction of P_2 to P_2 then P_2 is NP- complete.

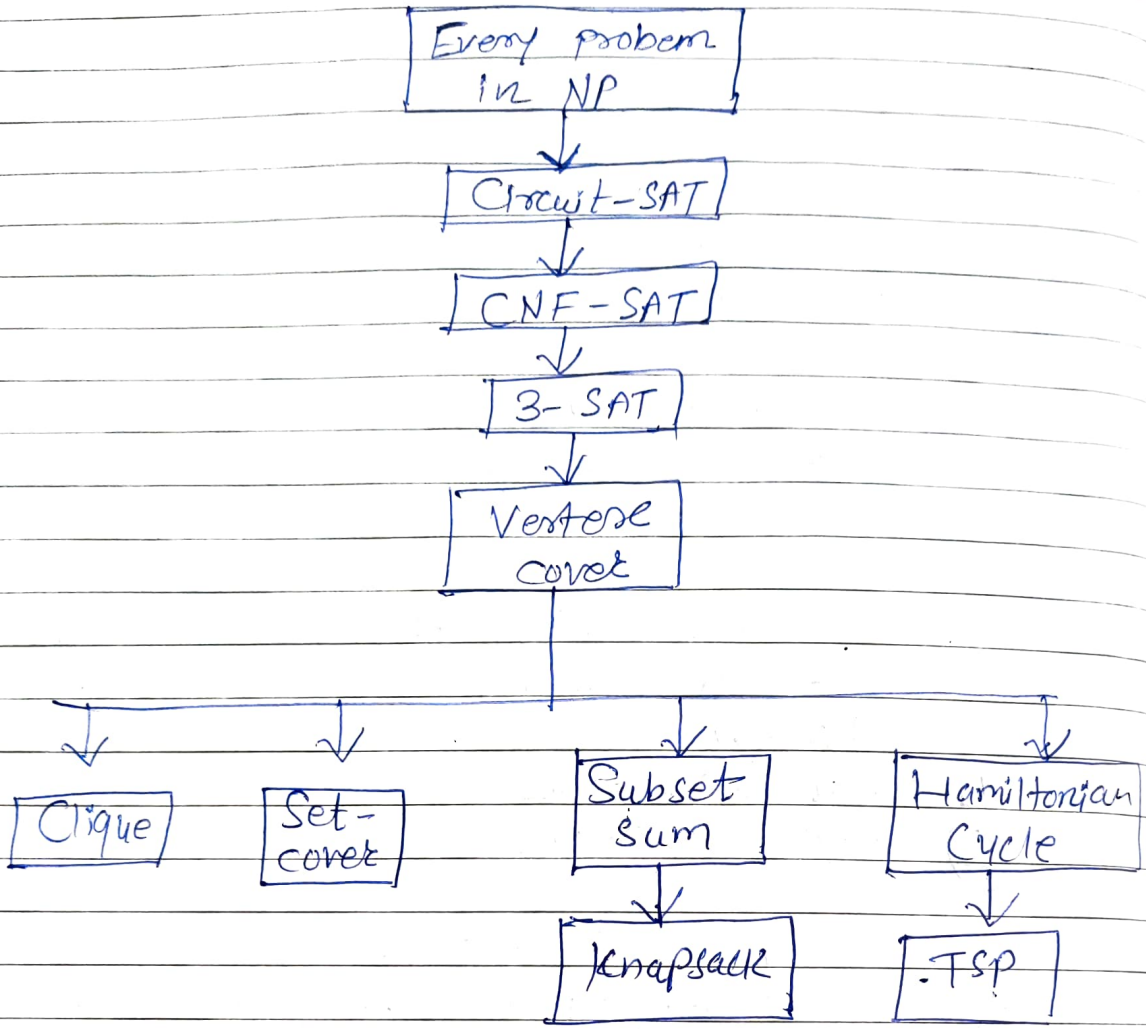


Fig ⇒
Reduction in NP-completeness

* NP-Complete problems
We are going to discuss 2 problems
1) Vertex cover
2) 3SAT.
And will prove that these 2 problems are NP-complete problems.

~~Steps for proving NP-complete~~

Steps to prove Problem L_2 is NP-Complete:

Step 1 - To show that L_2 is NP-hard

Step 2 - An NP-hard decision problem L_2 can be shown to be NP-complete by exhibiting a polynomial time non-deterministic algorithm for L_2.

How to Prove Problem is NP-hard:

Step 1 - pick a problem L_1 already known to be NP-hard.

Step 2 - Show how to obtain (in Polynomial deterministic time) an instance I' of L_2 from any instance I of L_1 such that from the solution of I' we can determine (in polynomial deterministic time) the solution to instance I of L_1

Step 3 - Conclude from step (2) that $L_1 \propto L_2$.

Step 4 - conclude from steps (1) and (3) and the transitivity of $\propto$ that L_2 is NP-hard.

Page No.
Date

3 SAT Problem:

Before this what is Satisfiability ?⇒

- Let x_1, x_2--- denote boolean variables
- $\bar{x}_i$ denote the negation of x_i.
- A literal is either a variable or its negation
- A formula in the propositional calculus is an expression that can be constructed using literals and with operations and, or.
- Symbol $\vee$ denotes or
  $\wedge$ denotes and.

- CNF (conjuctive Normal form) is formula where like

$$(x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

where each clause is conneted using $\wedge$ and literals using $\vee$.

- Satisfiability problem is to determine whether a formula is true for some assignment of truth values to the variables.

What is 3SAT problem?

- 3-SAT problem is the problem of determining the satisfiability of a CNF formula where each clause is limited to at most 3 literals.

Page No.
Date

3-SAT is NP-complete

Proof.

1) 3-SAT is NP. Given an assignment, we can just check that each clause is covered.

2) - 3-SAT is hard. To prove this, a reduction from SAT to 3-SAT must be provided.
- we will transform each clause independently based on its length.

— Reduction SAT to 3-SAT.

• Suppose a clause contains K literals.

(1) if K=1 (meaning $C_i = \{z_1\}$) we can add 2 new variables v_1 & v_2 and transform this into 4 clauses.

$$\{v_1, v_2, z_1\} \{v_1, \bar{v}_2, z_1\} \{\bar{v}_1, v_2, z_2\}$$
$$\{\bar{v}_1, \bar{v}_2, z_1\}.$$

(2) If K=2 ($C_i = \{z_1, z_2\}$) we can add in one variable v_1 and 2 new clauses

$$\{v_1, z_1, v_2\} \quad \{\bar{v}_2, z, z_2\}$$

(3) if K=3 ($C_i = \{z_1, z_2, z_3\}$), we move the clause as it is.

(4) if K>3 ($C_i = \{z_1, z_2, \ldots z_K\}$) we can add in K-3 new variables ($v_1, \ldots v_{K-3}$) and K-2 clauses:

$\{z_1, z_2, v_1\}$ $\{\overline{v_1}, z_3, v_2\}$ $\{\overline{v_2}, z_4, v_3\}$

..... $\{\overline{v_{k-3}}, z_{k-1}, z_k\}$

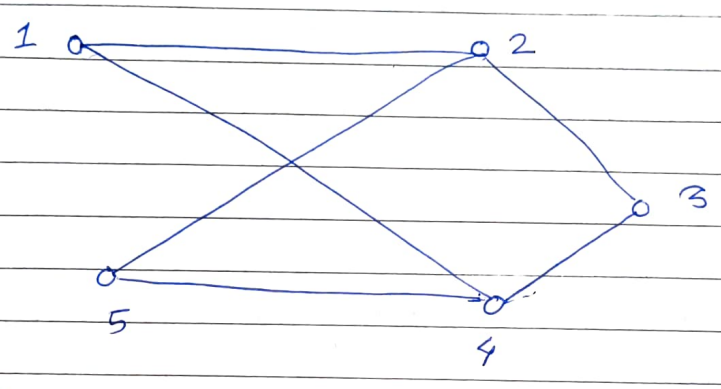
- One of the x_i must be true for the original clause to be true.
- It can be seen that, if the original clause is satisfiable than its equivalent 3SAT representation is satisfiable too.
- Since any SAT solution will satisfy the 3-SAT instance and a 3-SAT soln can set variables giving a SAT soln, the problem are equivalent.
- As SAT instance can be reducable to 3-SAT. so 3SAT is also NP hard.
- As 3-SAT is NP as well NP hard so we can say that 3SAT is NP complete

Page No.
Date

Prove that Vertex cover is NP-complete (Node cover)

- A Set $S \subseteq V$ in ~~node~~ vertex cover for a graph $G = (V, E)$ if and only if all edges in E are incident to at least one vertex in S.

- The size $|S|$ of the cover is the number of vertices in S.



In above example
- $S = \{2, 4\}$ is a vertex cover of size 2.
- $S = \{1, 3, 5\}$ is a vertex cover of size 3.

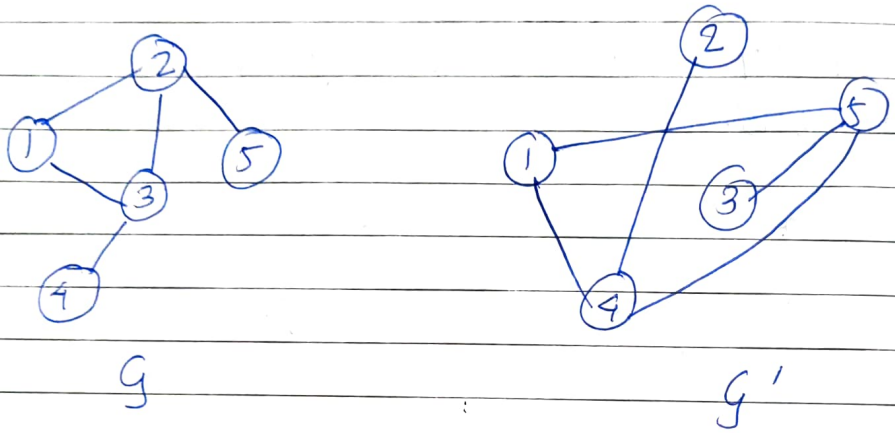
→ In vertex cover decision problem we are giving a graph G and an integer K. We are required to determine whether G has a node cover of size at most K.

Theorem ⇒ vertex cover is NP complete.

proof ⇒
- ⇒ Let $G = (V, E)$ and K define an instance of CDP. (clique decision problem)
- ⇒ Assume that $|V| = n$.
- ⇒ We construct a graph G' such that G' has a node cover of size at most $n-K$. if and only if G has a clique of size at least K.

- Graph ~~G' is~~ G' is given by $G' = (V, \bar{E})$ where $\bar{E} = \{(u,v) \mid u \in V, v \in V$ and $(u,v) \notin E\}$.
- The set G' is known as the complement of G.
- G' can be obtained from G in polynomial time.

Example -



G G'

Above figure shows a Graph G and its complement G'. In this figure, G' has vertex cover of {4,5}, since every edge of G' is incident either on the node 4 or on the node 5.

→ Thus G has a Clique of size 5-2 = 3, consisting of the nodes 1, 2, 3.

→ Since ~~CDP & Vertex cover~~ CNF Satisfiability (3SAT) ∝ CDP ∝ Vertex Cover we can say that Vertex cover is also NP hard.

→ Vertex cover is also in NP because we can nondeterministically choose a subset $C \subseteq V$ of size k and verify in polynomial time that C is a Cover of G.

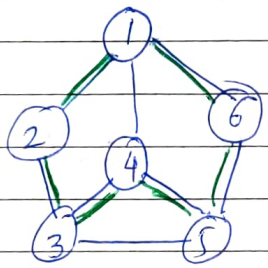
→ So Vertex Cover is NP-complete.

Page No.
Date

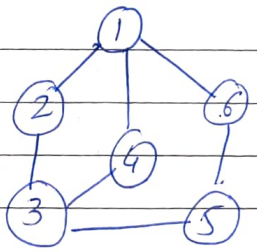
Hamiltonian Cycle Problem :-

- Hamiltonian cycle or ~~Eu~~ circuit of a given graph is round-trip path that starts at a specific node, visits all nodes excluding the starting node in the graph exactly once, and ends at the starting node.
- A graph can have zero, one or multiple Hamiltonian cycles in it.
- It applies to both directed & undirected graph.
- In Travelling Salesperson problem (TSP), a tour completed by a Salesperson is a Hamiltonian cycle.

eg.



G with Hamiltonian cycle (1-2-3-4-5-6-1) and (1-6-5-4-3-2-1)



G with no Hamiltonian cycle

Theorm ⇒ Hamiltonian cycle is NP hard.
We show SAT α DHC. (Directed Hamiltonian cycle)

Proof ⇒ Let F be a propositional formula in CNF.
- We will construct a directed Graph G from formula F.

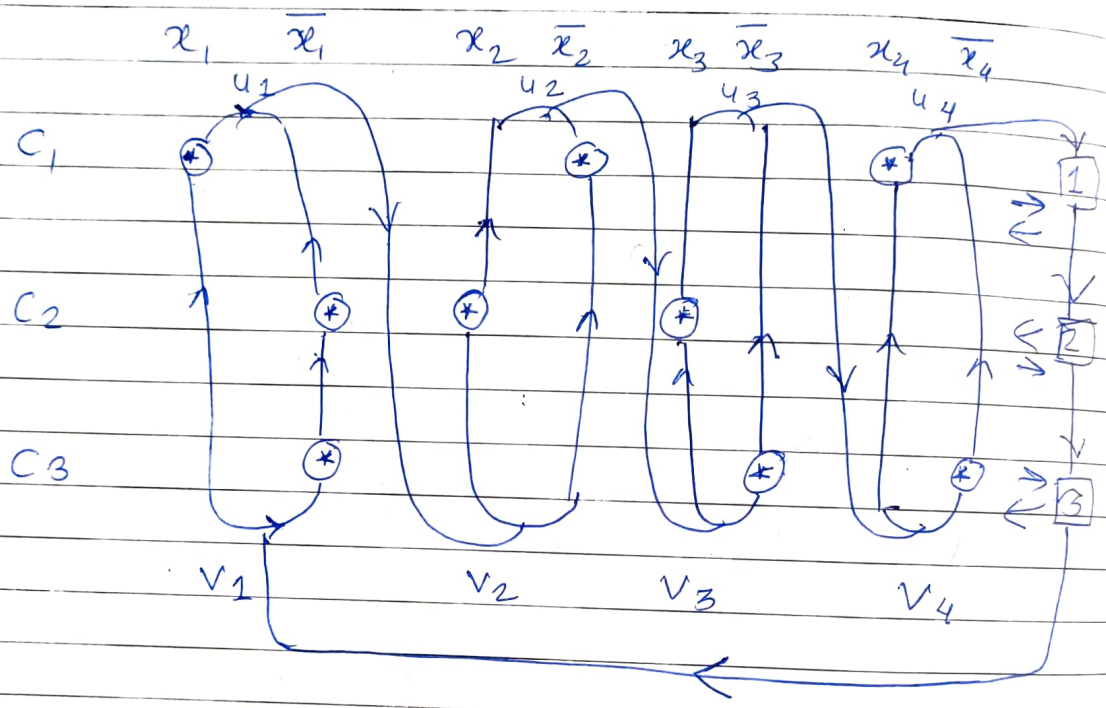
eg. $F = C_1 \wedge C_2 \wedge C_3$

$C_1 = x_1 \vee \overline{x_2} \vee x_4$

$C_2 = \overline{x_1} \vee x_2 \vee x_3$

$C_3 = x_1 \vee \overline{x_3} \vee \overline{x_4}$

- Assume that F has r clauses $C_1, C_2, \ldots C_r$ and n variables $x_1, x_2 \ldots x_n$.
- Draw an array with r rows and 2n columns.
- Row i denotes clause C_i.
- Each variables x_i is represented by two adjacent columns, one for each of the literals x_i and $\bar{x}_i$.



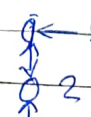
⇒ Insert a ⊛ into column x_i & row C_j if & only if x_i is a literal in C_j. Same follow for $\bar{x}_i$

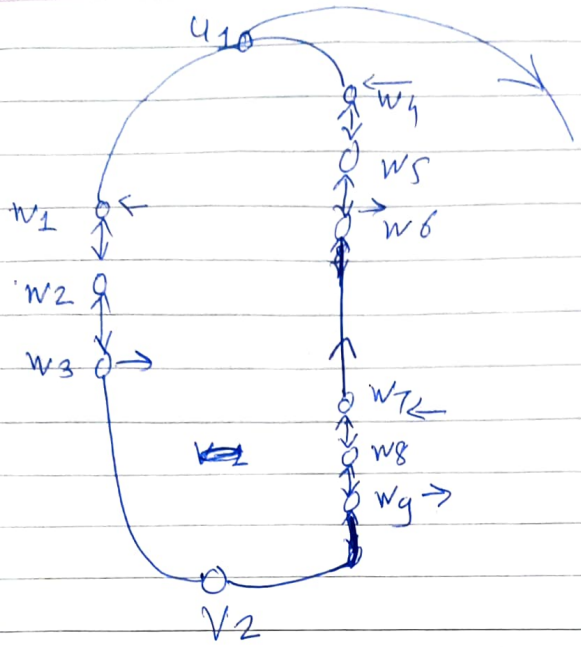
⇒ Betⁿ each pair of columns x_i and $\bar{x}_i$ introduce two vertices u_i and v_i. u_i at top & v_i at bottom.

⇒ Draw two chains of edges upward from v_i to u_i, one connecting together all ⊛s in column x_i and the other connecting all ⊛s in column $\bar{x}_i$.

⇒ Now draw edges $\langle u_i, v_{i+1} \rangle$.

⇒ To complete the graph, replace ⊛ by a subgraph





→ A Hamiltonian cycle for Graph G can start at v_1 & go to u_1, then to v_2 --- u_2 — v_3 — u_3 --- v_4 — u_4 — upto u_n.

→ In going from v_i to u_i, this cycle uses the column corresponding to x_i if x_i is true in S. otherwise it goes up the column to $\overline{x_i}$.

→ From u_n this cycle goes through (1), (2)— & come back to v_1.

→ As we can construct HC graph from SAT problem, we can say that

SAT $\propto$ HC

→ Hence HC is also NP hard.